

PL/SQL Programming Exercises

Name: Rakesh N

Program: Cognizant Digital Nuture 5.0

Exercise 1: Control Structures

Scenario 1: Apply 1% Interest Discount

Problem Analysis: Senior customers (>60 years) should receive a 1% interest discount on loans.

Root Cause: Loan interest rates are not automatically adjusted for eligible customers.

Solution: Loop through the Customers table, check age, update the InterestRate in Loans, and commit changes.

Expected Output:

Discount applied for Customer 101

Discount applied for Customer 105

Prevention Tips: Validate customer age before updating and always commit only after successful updates.

Scenario 2: Promote Customers to VIP

Problem Analysis: Customers with balances greater than 10000 should become VIP customers.

Root Cause: VIP status is not updated automatically based on account balance.

Solution: Iterate through customers, check balance, update IsVIP='TRUE', then commit.

Expected Output:

Customer 201 promoted to VIP

Customer 203 promoted to VIP

Prevention Tips: Validate balance values and avoid unnecessary updates.

Scenario 3: Loan Due Reminders

Problem Analysis: Notify customers whose loans are due within the next 30 days.

Root Cause: Customers may miss payment deadlines without reminders.

Solution: Select loans due within 30 days and print reminder messages using DBMS_OUTPUT.PUT_LINE.

Expected Output:

Reminder: Loan 10 for Customer 301 is due on 02-JUL-2026

Reminder: Loan 11 for Customer 302 is due on 15-JUL-2026

Prevention Tips: Schedule reminder procedures to run daily.

Exercise 3: Stored Procedures

Scenario 1: ProcessMonthlyInterest

Problem Analysis: Apply monthly interest to all savings accounts.

Root Cause: Account balances remain unchanged without automated processing.

Solution: Create a stored procedure that updates all balances by 1% and commits.

Expected Output:

Monthly interest applied to all savings accounts.

Prevention Tips: Execute the procedure only once per billing cycle.

Scenario 2: UpdateEmployeeBonus

Problem Analysis: Increase employee salaries department-wise using a bonus percentage.

Root Cause: Manual salary updates are error-prone.

Solution: Create a procedure accepting department and bonus percentage, update salaries, and commit.

Expected Output:

Bonus of 10% applied to employees in Sales department.

Prevention Tips: Validate bonus percentages before applying updates.

Scenario 3: TransferFunds

Problem Analysis: Transfer money safely between two accounts.

Root Cause: Transfers should not proceed when the source account has insufficient balance.

Solution: Check source balance, perform debit and credit operations, then commit; otherwise display an error.

Expected Output:

Transfer of 5000 completed from Account 101 to Account 102.

Prevention Tips: Always validate balances and use transactions to maintain consistency.

Conclusion

PL/SQL control structures and stored procedures help automate repetitive database tasks, improve data integrity, and simplify business logic by using loops, conditional statements, and reusable procedures.